Software Re-Engineering

Lecture 12

Vagrant with Ansible

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

- Though Vagrant handles the generation and basic imaging of virtual machines to form isolated development environments, it doesn't do the provisioning of these virtual machines.
- Provisioning is the concept of preparing and equipping a machine to allow it to provide (new) services to its users
- So, it's up to us to handle the installation and configuration of the packages we want to work.
- Fortunately, Vagrant easily integrates with existing provisioners such as Chef, Puppet, and Ansible.



- Once a virtual machine is generated and a provisioner is ready, we can quickly start a development environment on our machine without having to worry about finding, installing, and configuring project dependencies.

- Any developer can pickup a project and start working on it with little work with the same environment as everyone else.



- Ansible communicates over normal SSH channels in order to retrieve information from remote machines, issue commands, and copy files.
- Ansible can interact with clients through either command line tools or through its configuration scripts called Playbooks.
- It uses a very simple language (YAML, in the form of Ansible Playbooks) that allow you to describe your automation jobs in a way that approaches plain English.
- YAML stands for Yet Another Markup Language.
- A *Playbook* is simply a text-file that defines which *roles* get assigned to which machines

What Problem It Solves:

NASA needed to move roughly 65 applications from a traditional hardware-based data center to a cloud-based environment for better agility and cost savings. The rapid timeline resulted in many applications being migrated ‘as-is’ to a cloud environment. This created an environment spanning multiple virtual private clouds (VPCs) and AWS accounts that could not be easily managed. Even simple things, like ensuring every system administrator had access to every server, or simple patching, were extremely burdensome.

NASA used the Ansible to manage and schedule the cloud environment.

As a result of implementing the Ansible, NASA is better equipped to manage its AWS environment.

This allowed NASA to provide better operations and security to its clients. It has also increased efficiency as a team.

- Updating nasa.gov went from over 1 hour to under 5 minutes
- Patching updates went from a multi-day process to 45 minutes
- Achieving near real-time RAM and disk monitoring
- Provisioning OS Accounts across an entire environment in under 10 minutes
- Application stack set up from 1–2 hours to under 10 minutes.

How Ansible works?

- In Ansible, there are two categories of computers: the ***control node*** and ***managed nodes*****.**
- The control node is a computer that runs Ansible.
- There must be at least one control node, although a backup control node may also exist.
- A managed node is any device being managed by the control node.



- Ansible works by connecting to nodes (clients, servers, or whatever you're configuring) on a network, and then sending a small program called an Ansible module to that node.
- Ansible executes these modules over SSH and removes them when finished.

- The only requirement for this interaction is that your Ansible control node has login access to the managed nodes.
- SSH keys are the most common way to provide access, but other forms of authentication are also supported.



- The units of code that Ansible executes on the managed nodes are called modules.
- Each module is invoked by a task, and an ordered list of tasks together forms a playbook.
- Users write playbooks with tasks and modules to define the desired state of the system.



Basic Concepts & Terms

Host: A remote machine managed by Ansible.

Group: Several hosts grouped together that share a common attribute.

Inventory: A collection of all the hosts and groups that Ansible manages.

Modules: Units of code that Ansible sends to the remote nodes for execution.

Tasks: Units of action that combine a module and its arguments along with some other parameters.

Playbooks: An ordered list of tasks along with its necessary parameters that define a recipe to configure a system.

Roles: Redistributable units of organization that allow users to share automation code easier.

YAML: A popular and simple data format that is very clean and understandable by humans



All YAML files (regardless of their association with Ansible or not) can optionally begin with --- and end with .... This is part of the YAML format and indicates the start and end of a document.

All members of a list are lines beginning at the same indentation level starting with a "- " (a dash and a space):

```
---
# A list of tasty fruits
- Apple
- Orange
- Strawberry
- Mango
...
```



A dictionary is represented in a simple key: value form (the colon must be followed by a space):

```
# An employee record
martin:
  name: Martin D'vloper
  job: Developer
  skill: Elite
```

More complicated data structures are possible, such as lists of dictionaries, dictionaries whose values are lists or a mix of both:

```
# Employee records
- martin:
    name: Martin D'vloper
    job: Developer
    skills:
      - python
      - perl
      - pascal
- tabitha:
    name: Tabitha Bitumen
    job: Developer
    skills:
      - lisp
      - fortran
      - erlang
```

you can also specify a boolean value (true/false) in several forms:

create_key: true
needs_agent: false
knows_oop: True
likes_emacs: TRUE
uses_cvs: false



What Are Inventories In Ansible?

- Static lines of servers
- Dynamic list of servers: AWS, Azure, GCP, etc.
- The headings in brackets are group names

```
1   ## Ansible Hosts File for Lab
2
3   [CSR-Routers]
4   CSR-1 ansible_host=192.168.1.210
5   CSR-2 ansible_host=192.168.1.211
6   CSR-3 ansible_host=192.168.1.212
7
8   [SWITCHES]
9   2960 ansible_host=192.168.1.250
```

Here’s that same basic inventory file in YAML format:

```
ungrouped:
  hosts:
    mail.example.com:
webservers:
  hosts:
    foo.example.com:
    bar.example.com:
dbservers:
  hosts:
    one.example.com:
    two.example.com:
    three.example.com:
```

Default groups

- Even if you do not define any groups in your inventory file, Ansible creates two default groups: **all** and **ungrouped.**
- The **all** group contains every host. The **ungrouped** group contains all hosts that don’t have another group aside from all.
- Every host will always belong to **at least 2 groups** (all and ungrouped or all and some other group).
- For example, in the basic inventory above, the host **mail.example.c**om belongs to the all group and the ungrouped group; the host **two.example.com** belongs to the all group and the dbservers group.

Ansible Roles

1.Roles are a way to group tasks together into one container. We could have a role for setting up MySQL, another one for configuring ip tables.

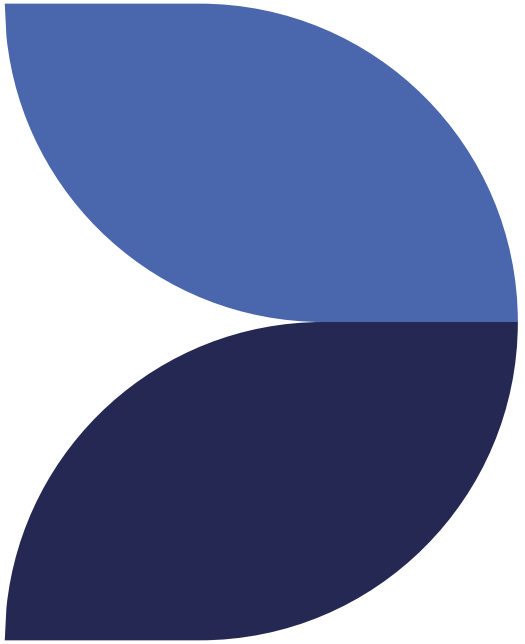
2.Roles make it easy to configure hosts. Any role can be performed on any host or group of hosts such as:

- *hosts: all*
- *roles:*
- *role_1*
- *role_2*



Create a Playbook

Let us start by writing a sample YAML file.



```
---
- hosts: all
  tasks:
    - name: Print message
      debug:
        msg: Hello Ansible World
```

- Tasks are defined as a list under the name tasks inside a play, at the same level as the hosts directive that defines the targets for that play. The name property defines the output that will be printed out when that task is about to be executed.

- The example task invokes the debug module, which allows you to display messages in a play. These messages can be used to show debug information such as the contents of a variable or the output message returned by a command, for instance.

- Each module has its own set of options and properties. The debug module expects a property named msg containing the message to be printed out. Pay special attention to the indentation (2 spaces), since msg must be a property inside debug.

```
---
- hosts: all
  vars:
    - username: YOUR_USERNAME
    - home: /home/YOUR_USERNAME
  tasks:
    - name: print variables
      debug:
        msg: "Username: {{ username }}, Home dir: {{ home
}}"
```

hosts: Specifies the target servers where the tasks will be executed.
become: yes: Instructs Ansible to run the tasks with elevated privileges (usually using sudo or root).
tasks: This section contains a list of tasks that Ansible will execute sequentially.

run the playbook using the following command.

$ansible-playbook <filename>.yaml

Installing Required Packages

git, nginx, sqlite3, mcrypt, and a couple of php5-* packages.

```
---
- hosts: php
  sudo: yes

  tasks:
  - name: install packages
    apt: name={{ item }} update_cache=yes state=latest
    with_items:
      - git
      - mcrypt
      - nginx
      - php5-cli
      - php5-curl
      - php5-fpm
      - php5-intl
      - php5-json
      - php5-mcrypt
      - php5-sqlite
      - sqlite3
```

```
---
- hosts: php
  sudo: yes
```

hosts: php

•This means the playbook will run on all hosts in the **php group** (as defined in your inventory file).

•Example: If your inventory has this:

```
[php]
192.168.1.100
192.168.1.101
```

An **inventory file** is where you define the list of **hosts (machines/servers)** that Ansible will manage. It tells Ansible *"These are the systems I want to run my playbooks on."*

Ansible inventory files can be in different formats:

Format	Common Extension	Description
INI	.ini or no extension	Simple format with groups and hostnames
YAML	.yml or .yaml	Preferred for dynamic or structured configuration
JSON	.json	Used in dynamic inventories or APIs

Static Inventory (INI Format)

```
[web]
192.168.1.10
192.168.1.11

[db]
dbserver1.example.com
dbserver2.example.com
```

Save it as:
inventory or inventory.ini

Static Inventory (YAML Format)

```
all:
  children:
    web:
      hosts:
        192.168.1.10:
        192.168.1.11:
    db:
      hosts:
        dbserver1.example.com:
        dbserver2.example.com:
```

Sudo: yes
Or become: yes

This line tells Ansible to run tasks as superuser (root) using sudo.

```
---
- hosts: php
  sudo: yes

  tasks:
   - name: install packages
     apt: name={{ item }} update_cache=yes
state=latest
     with_items:
       - git
       - mcrypt
       - nginx
       - php5-cli
       - php5-curl
       - php5-fpm
       - php5-intl
       - php5-json
       - php5-mcrypt
       - php5-sqlite
       - sqlite3
```

tasks:This begins the list of actions (tasks) that Ansible will perform on the target hosts.

- name: install packages

This is a human-friendly description of the task.

```
apt: name={{ item }} update_cache=yes
state=latest
    with_items:
      - git
      - mcrypt
      - nginx
      - php5-cli
      - php5-curl
      - php5-fpm
      - php5-intl
      - php5-json
      - php5-mcrypt
      - php5-sqlite
      - sqlite3
```

•This task uses the **apt module**, which is used to install packages on Debian/Ubuntu-based systems.

Breakdown of parameters:

•name={{ item }}: Installs the current item in the loop (see with_items below).

•update_cache=yes: Runs apt-get update before installing to get the latest package lists.

•state=latest: Ensures the package is installed and **up to date** with the latest version.

with_items:

•This is a **loop** in Ansible.

•It means: **Repeat this task for each item in the list** below.

Let's suppose we want to install the latest OpenSSL version in our ubuntu server. these are the command we would execute in our server

```
apt update
apt install openssl
(or)
apt-get update
apt-get install openssl
```

With an ansible apt module,

```
---
 - name: Ansible apt module examples
   hosts: web
   become: true
   tasks:
    - name: Ansible Update Cache and install openssl
      apt:
        name: openssl
        state: present
        update_cache: yes
```

An outdated cache can lead apt to miss updates or to fail to install a package altogether (because the version it wants is no longer available from the repositories).

update_cache=yes tells Ansible's apt module to refresh the caches before applying whatever change is necessary (if any).

Let us suppose we want to upgrade the OpenSSL version installed in our system and this is the command we would execute on our servers

```
# Update the Cache
apt update
# Upgrade the package openssl
apt upgrade openssl (or)
apt-get  update
apt-get upgrade openssl
```

```
---
 - name: Ansible apt module examples
   hosts: web
   become: true
   tasks:
    - name: Ansible Update Cache and upgrade openssl
      apt:
        name: openssl
        state: latest
        update_cache: yes
```

If you have not noticed it already. the **state: latest** parameter would make sure that the package is upgraded and stay the latest.

This is the playbook we are going to use to install / setup a LAMP (LinuxApacheMysqlPhp) stack on our Debian/Ubuntu server

```
- name: Ansible apt module examples
  hosts: web
  become: yes
  tasks:
    - name: Ansible apt to install multiple packages - LAMP
      register: updatesys
      apt:
        update_cache: yes
        name:
          - firewalld
          - apache2
          - mariadb-server
          - php
          - php-mysql
        state: present
```

let's see how to specify the package version with ansible apt module.

```
---
 - name: Ansible apt module examples
   hosts: web
   become: true
   tasks:
    - name: Ansible apt install tomcat Version 8.0.32
      register: updatesys
      apt:
        update_cache: yes
        name: tomcat8=8.0.32-1ubuntu1.11
        state: present
```

here is the sample playbook to validate if few packages are installed or not.

```
---
 - name: Ansible validate if the packages are installed
   hosts: web
   become: true
   tasks:

    - name: Gather Package facts
      package_facts:
        manager: auto

    - name: Validating if the package is installed or not
      debug:
        msg: "{{item}} is installed"
      when: '"{{item}}" in ansible_facts.packages'
      with_items:
        - apache2
        - nginx
        - filebeat
```

prints only apache2 is installed

It clears the useless packages from cache and remove the dependency tools which are no longer required.

```
- name: Remove useless packages from the cache
  apt:
    autoclean: yes

- name: Remove dependencies that are no longer required
  apt:
    autoremove: yes
```

```
- name: Clone git repository
  git: >
    dest=/var/www/laravel
    repo=https://github.com/laravel/laravel.git
    update=no
  sudo: yes
  sudo_user: www-data
```

Loops

Ansible allows us to iterate over a set of items in a task to execute it multiple times with different parameters without rewriting it. For example, to create several files, we would use a task that iterates over a list of directory names instead of writing five tasks with the same module.

To iterate over a simple list of items, use the loop keyword. We can reference the current value with the loop variable item.

```
- name: "Create some files"
  ansible.builtin.file:
    state: touch
    path: /tmp/{{ item }}
  loop:
    - example_file1
    - example_file2
    - example_file3
```

It’s also possible to iterate over dictionaries:

```
- name: "Create some files with dictionaries"
  ansible.builtin.file:
    state: touch
    path: "/tmp/{{ item.filename }}"
    mode: "{{ item.mode }}"
  loop:
    - { filename: 'example_file1', mode: '755'}
    - { filename: 'example_file2', mode: '775'}
    - { filename: 'example_file3', mode: '777'}
```

Install and configure a web server

The first thing that the automation should handle is the installation and configuration of NGINX, the chosen web server. The **roles/webserver/tasks/main.yml** file shown below is a straightforward way to accomplish this:

```
---
- name: Install packages
  package:
    name: "{{ webserver_packages }}"
    state: latest

- name: Add base NGINX configuration file
  copy:
    dest: /etc/nginx/nginx.conf
    src: etc/nginx/nginx.conf
    owner: root
    group: root
    mode: 0644
  notify: Reload NGINX
```

The webserver_packages variable is defined in the **roles/webserver/vars/main.yml file**:

```
---

webserver_packages:
  - nginx
```

State as '**Present**' and '**Installed**' are used interchangeably. They both do the same thing i.e. it will ensure that a desired package in your case 'yum' is installed.

Whereas State as '**Latest**' means in addition to installation, it will go ahead and update if it is not of the latest available version.

Whenever you are building your stack/app or working on production, it is always advisable to use '**Present**' or '**Installed**' state.

- **Vagrant** uses a Vagrantfile, which is written in **Ruby**.
- **Ansible** uses .yml or .yaml files, which follow the **YAML** syntax.

Example: Vagrant calling Ansible

Here's a minimal Vagrantfile (Ruby) that uses Ansible (YAML) for provisioning:

Vagrantfile

ruby

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/focal64"

  config.vm.provision "ansible" do |ansible|
    ansible.playbook = "provision.yml"
  end
end
```

provision.yml (Ansible playbook)

yaml

```
---
- name: Provision VM with Apache
  hosts: all
  become: yes
  tasks:
    - name: Install Apache
      apt:
        name: apache2
        state: present
        update_cache: yes
```

What happens?

1. vagrant up — starts a VM using the Ubuntu box
2. Vagrant checks the provisioner (here, Ansible)
3. Vagrant runs ansible-playbook provision.yml automatically **inside or against the VM**
4. Ansible provisions the VM — installing Apache in this case

To Dos

Try deploying simple web application on some web server using ansbile

Docker

- Docker is an open platform that enables users to develop, ship, and run applications with ease.
- Docker software is packaged as containers- a docker standardized unit.
- These containers have all the elements, such as system tools, libraries, runtime, etc required by the software.
- Docker container is an executable package of applications and its dependencies bundled together



- Difference from virtual machine:
  - With a virtual machine, **the memory usage is very high**, whereas, in a Docker environment, memory usage is very low.
  - In terms of portability, virtual machines just are not ideal. They’re still dependent on the host operating system, and a lot of problems can happen when you use virtual machines for **portability**.
  - In contrast, Docker was designed for portability. You can actually build solutions in a Docker container, and the solution is guaranteed to work as you have built it no matter where it’s hosted.



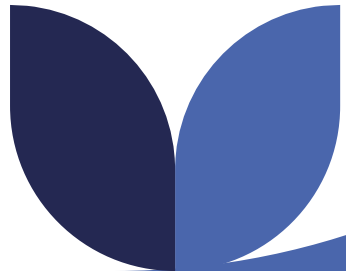
- One of the other challenges of using a virtual machine is that if you have unused memory within the environment, you cannot reallocate it.
- If you set up an environment that has 9 gigabytes of memory, and 6 of those gigabytes are free, you cannot do anything with that unused memory.
- With Docker, if you have free memory, you can reallocate and reuse it across other containers used within the Docker environment.
- The boot-up time for a virtual machine is about a few minutes, in contrast to the milliseconds it takes for a Docker environment to boot up.



Vagrant is more resource heavy

- Because Vagrant uses existing virtualization software under the hood (it can work on top of VirtualBox, VMWare and Hyper-V among others), it seems that Docker has an advantage in being more lightweight as it uses Linux kernel features instead of hypervisors.
- But if it requires Linux, how come Docker also runs on Windows and macOS? As it turns out, Docker Desktop for Windows and for Mac use virtualization (Hyper-V and HyperKit respectively) to run the Linux kernel, so it makes Docker lose its lightweight appeal

Vagrant is available on all major platforms i.e. Windows, macOS and Linux, and its installation is fairly straightforward. However, it requires us to have a hypervisor already installed - VirtualBox is probably the best option as it's free, open-source, well supported and runs on all previously mentioned systems



Why Vagrant?

Ideal for Legacy Applications

"Vagrant works better when dealing with legacy software that can’t be containerized easily or doesn’t run well in minimal Linux containers."

Many **older/legacy applications** require:

- A full OS environment
- Specific kernel versions or systemd
- GUI components or services that don’t run well in containers

Example:
You’re working on a legacy **Java web app** from 2009 that:
•Requires **Ubuntu 16.04**
•Needs **Apache2**, **Java 7**, and an old **MySQL 5.5**
•Has **init.d** scripts instead of systemd

Docker Struggles:
•Alpine or slim containers don’t support init.d properly
•Finding or building a container image with Java 7 and MySQL 5.5 is painful
•OS-level services don’t behave normally inside a container

Vagrant Solution

You now have a **real Ubuntu 16.04 VM**, behaving just like a physical server.

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/xenial64"
  config.vm.provision "shell", inline: <<-SHELL
    sudo apt-get update
    sudo apt-get install -y openjdk-7-jdk mysql-server apache2
  SHELL
end
```

<<-SHELL ... SHELL

Everything between <<-SHELL and SHELL is treated as a multi-line string.The name SHELL is just a label — it could be SCRIPT, EOF, or anything.The important part is that the opening and closing marker must match.

This is a **Ruby feature** called a **"heredoc"**

```
<<-MARKER
  multi-line string content
MARKER
```

telling Vagrant:
“Run these shell commands **inside the VM** right after it's created.”

Isolation from Host System

"Vagrant VMs are fully isolated; nothing leaks into your host system unless you configure shared folders. Docker shares the host kernel, which can introduce conflicts."

- Docker containers share your host’s kernel and networking
- This can lead to port conflicts, file permission issues, or even security concerns
- Vagrant, by using VirtualBox or another hypervisor, gives each VM its own virtual hardware (memory, disk, network, etc.)

Example:
You have two projects:
•Project A runs **PostgreSQL 13**
•Project B runs **PostgreSQL 9.6** (old version)

Docker Risk:
Running both containers on Docker on the same host:
•Might conflict on ports (e.g., both want port 5432)
•Might use shared volumes with mismatched permissions
•Share same kernel — risky if an image has an exploit

Vagrant Solution:

Each project runs in its **own isolated VM**:

```
# Project A - PostgreSQL 13
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/focal64"
  config.vm.network "private_network", ip: "192.168.56.10"
  config.vm.provision "shell", inline: <<-SHELL
    sudo apt install -y postgresql-13
  SHELL
end
```

```
# Project B - PostgreSQL 9.6
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"
  config.vm.network "private_network", ip: "192.168.56.11"
  config.vm.provision "shell", inline: <<-SHELL
    sudo apt install -y postgresql-9.6
  SHELL
end
```

Each VM has its own kernel, IP, and environment. No port clash, no kernel issues.